

Stochastic Processes

Semester: 14042

Lecturer: Dr. Peyvandi



Project Report II

Students: Mobin Yousefi, Mehrshad Haghighat

IDs: 402100594, 402100418

Introduction

The other report we wrote covers what the project is about and what results we got. This one is different on purpose: instead of talking about the concepts, we go function by function through our code and explain the coding decisions behind it — why we structured a computation a certain way, why we added a particular safety check, why we picked one numpy operation over another, and what would go wrong if we hadn't.

Why NumPy?

Before getting into individual functions, it's worth explaining why we built the whole project on top of NumPy in the first place, instead of just writing everything in plain Python.

- **Every core object in this project is a vector or a matrix.** A GP prior sample is a vector of function values at a grid of points; a kernel is an $n \times n$ covariance matrix; GP regression is entirely matrix algebra (products, solves, Cholesky factors); even the MCMC and Langevin samplers, which look like simple scalar loops, repeatedly rebuild and factor covariance matrices inside every iteration. None of this is naturally a list-of-numbers problem, it's a linear-algebra problem, and NumPy is what gives Python real array types (`ndarray`) with the matrix operations (`@`, `.T`, broadcasting) that this kind of math is written in.
- **Speed.** A pure-Python double loop over every pair of points to build a 200×200 kernel matrix means 40,000 Python-level float operations, and Python's interpreter overhead per operation is large. NumPy pushes that loop down into compiled C code and lets us express it as a handful of array operations instead (see the `rbf_kernel` vectorization in Task 1) — the exact same computation runs orders of magnitude faster with no algorithmic change, just because it's not being interpreted one scalar at a time anymore.
- **It's the natural vocabulary for the linear algebra we needed.** Sampling a multivariate Gaussian, computing a matrix inverse or a Cholesky factor, taking a log-determinant safely with `slogdet`, solving triangular systems, drawing i.i.d. standard normals for Langevin noise — all of these are one-line calls in NumPy (`np.random.multivariate_normal`, `np.linalg.inv`, `np.linalg.slogdet`, `np.random.randn`, etc.) instead of algorithms we'd have to implement and debug ourselves from scratch. Writing our own Cholesky decomposition or our own multivariate-normal sampler was never really the point of this assignment — understanding *when* and *why* to use them was, so leaning on well-tested NumPy/SciPy implementations let us focus our own code on the modeling and inference logic instead of on re-deriving numerical

linear algebra that already exists, and is already more carefully tested than anything we'd write ourselves in the time we had.

- **It's also what SciPy is built on.** Once we needed something NumPy itself doesn't provide directly (a proper Cholesky-based linear solve, in Task 2's `scipy.linalg.cho_factor/cho_solve`), SciPy's arrays are just NumPy arrays underneath, so nothing about the rest of the code had to change to accommodate it — `K_xx` built by our own `rbf_kernel` function feeds directly into `scipy.linalg` calls with no conversion step, because both libraries speak the same array format.

In short: NumPy isn't an incidental choice of library here, it's the reason the rest of the code in this report can be short, vectorized, and reasonably fast, instead of a much longer and slower pile of nested Python loops doing the same math by hand.

1 Task 1 — Kernel Code

1.1 Vectorizing the RBF kernel instead of looping

Our first instinct for the RBF kernel would have been a double `for` loop over every pair of points. We didn't do that, because for a 200×200 grid that's 40,000 Python-level iterations, which is slow and, more importantly, unnecessary. Instead we used the expansion

$$\|x_i - x_j\|^2 = \|x_i\|^2 + \|x_j\|^2 - 2x_i x_j$$

so the whole pairwise squared-distance matrix can be built from array broadcasting and one matrix multiplication:

```
1 def rbf_kernel(x1, x2, sigma=1.0, length_scale=1.0):
2     x1 = np.atleast_2d(x1).T
3     x2 = np.atleast_2d(x2).T
4
5     sqdist = np.sum(x1**2, axis=1).reshape(-1, 1) \
6             + np.sum(x2**2, axis=1) - 2 * x1 @ x2.T
7
8     return sigma**2 * np.exp(-sqdist / (2 * length_scale**2))
```

A few specific choices here:

- **`np.atleast_2d(x1).T`:** `x1` and `x2` come in as plain 1D arrays. We reshape them to column vectors so that `x1 @ x2.T` actually gives an (n, m) outer-product-style matrix instead of a scalar (a 1D array times its own transpose in numpy doesn't do what you'd want here).
- **`.reshape(-1, 1)` on the first sum but not the second:** we need `np.sum(x1**2, axis=1)` (shape $(n,)$) to broadcast down the *rows*, so we reshape it into a column $(n, 1)$. The second term, `np.sum(x2**2, axis=1)` (shape $(m,)$), already broadcasts correctly across *columns* without reshaping. Getting this backwards is a very easy bug to introduce — it doesn't crash, it just silently gives you the wrong matrix, so we double-checked the shapes here explicitly rather than trusting broadcasting blindly.
- **Why not just call `scipy.spatial.distance.cdist`?** We could have, but writing it out ourselves made it much easier to later reuse the exact same squared-distance logic inside the MCMC log-posterior in Task 3, where we call `rbf_kernel` again with a different length-scale

on every iteration — having full control over the function meant we knew exactly what it was doing when we started debugging why `slogdet` was occasionally returning a negative sign (more on that in Task 3).

1.2 The Brownian kernel is a single `np.minimum` call

```
1 def brownian_kernel(x1, x2):
2     x1 = np.atleast_2d(x1).T
3     x2 = np.atleast_2d(x2).T
4     return np.minimum(x1, x2.T)
```

This one is short because the whole derivation — $k(s, t) = \min(s, t)$ — collapses to a single broadcasted numpy call once `x1` is a column vector and `x2.T` is a row vector: `np.minimum` broadcasts them against each other into the full (n, m) matrix automatically. We reused the exact same `atleast_2d(...).T` reshaping trick as in `rbf_kernel` for consistency, even though a 1D version would technically work here too with careful indexing — keeping the two kernel functions structurally parallel made it easier to swap one for the other later without touching any calling code.

1.3 Why we add jitter before sampling

```
1 K_rbf += 1e-8 * np.eye(len(x_rbf))
2 K_brownian += 1e-8 * np.eye(len(x_brownian))
3
4 samples_gaussian = np.random.multivariate_normal(np.zeros(len(x_rbf)), K_rbf, size=5)
5 samples_brownian = np.random.multivariate_normal(np.zeros(len(x_brownian)), K_brownian
, size=5)
```

`np.random.multivariate_normal` does a Cholesky (or eigen-) decomposition of the covariance matrix internally, and that decomposition requires the matrix to be (numerically) positive definite. Our kernel matrices are positive *semi*-definite in theory, but floating-point rounding can push the smallest eigenvalue just barely negative, which makes the decomposition throw an error or produce garbage. Adding a tiny 10^{-8} to the diagonal nudges every eigenvalue up by that amount, which is small enough to not visibly change the samples but large enough to guarantee numerical positive-definiteness. We added this *after* building the full kernel matrix rather than baking it into `rbf_kernel`/`brownian_kernel` themselves, since we didn't want a hidden fudge factor silently changing the kernel definition everywhere it's used (e.g. inside the MCMC likelihood in Task 3, where the noise variance already handles this role).

2 Task 2 — GP Regression Code

2.1 Why the noise variance only goes into `K_xx`

```
1 K_xx = rbf_kernel(x_train, x_train) + noise_var * np.eye(len(x_train))
2 K_xs = rbf_kernel(x_train, x)
3 K_ss = rbf_kernel(x, x)
```

We only add `noise_var * np.eye(...)` to `K_xx`, not to `K_xs` or `K_ss`. This isn't arbitrary: the noise term models the fact that our *observations* $y_i = f(x_i) + \epsilon_i$ are noisy versions of the underlying function, so it only belongs on the covariance of the observed training points with themselves. `K_ss`

is the covariance of the latent function f at test points with itself (we want to predict the noise-free function there, not a noisy version of it), and K_{xs} is a cross-covariance between two different kinds of variables (y at training points, f at test points), so neither should carry the observation-noise term. Putting it in the wrong place is a common mistake that makes the posterior variance either too small (if omitted from K_{xx}) or too large everywhere (if mistakenly added to K_{ss} as well).

2.2 Why we switched from `np.linalg.inv` to a Cholesky-based solve

Our first working version of this cell (and the version we describe in words in the notebook comments as the simple fallback) just formed the explicit inverse:

```
1 # K_inv = np.linalg.inv(K_xx)
2 # mu_post = K_xs.T @ K_inv @ y_train
3 # cov_post = K_ss - K_xs.T @ K_inv @ K_xs
```

We kept that version in the notebook as a commented-out fallback, but replaced the actual computation with a Cholesky-based solve using `scipy.linalg`, because that's the numerically preferred way to do GP regression (this is the standard algorithm from Rasmussen & Williams' *Gaussian Processes for Machine Learning*, Algorithm 2.1) and we wanted to actually implement it ourselves instead of just leaving it as a comment:

```
1 import scipy.linalg as sys_linalg
2 ...
3 L = sys_linalg.cho_factor(K_xx, lower=True)
4 alpha = sys_linalg.cho_solve(L, y_train)
5 mu_post = K_xs.T @ alpha
6
7 v = sys_linalg.solve_triangular(L[0], K_xs, lower=True)
8 cov_post = K_ss - v.T @ v
```

Line by line, here's why each piece is written this way:

- **`cho_factor(K_xx, lower=True)` instead of writing our own Cholesky loop:** this is the same factorization idea we already used conceptually (Task 1 relies on `np.random.multivariate_normal` doing one internally), but here we call it explicitly because we need to reuse the *same* factor L (where $K_{xx} = LL^\top$) for both the mean and the covariance term below, instead of paying for a fresh $O(n^3)$ decomposition twice. `cho_factor` doesn't return a plain matrix, it returns a tuple `(c, lower)` — `c` is the packed triangular factor and `lower` just records whether it's lower- or upper-triangular, so that `cho_solve` and `solve_triangular` know how to interpret it later. That's why we index into it as `L[0]` when we need the raw matrix for `solve_triangular`.
- **`alpha = cho_solve(L, y_train)` instead of `K_inv @ y_train`:** mathematically $\alpha = K_{xx}^{-1}y$, but `cho_solve` never actually forms K_{xx}^{-1} — it solves the two triangular systems $Lz = y$ and $L^\top \alpha = z$ directly. Solving a linear system this way is both cheaper and more numerically stable than computing a full inverse and then multiplying, because forming an explicit inverse first amplifies rounding error unnecessarily (the classic numerical linear algebra advice: “don't invert a matrix just to multiply by a vector, solve the system instead”). We then get $\mu_* = K_{xs}^\top \alpha$ in one matrix-vector product.
- **`v = solve_triangular(L[0], K_xs, lower=True)` followed by `K_ss - v.T @ v`:** this replaces $K_{xs}^\top K_{xx}^{-1} K_{xs}$ with an equivalent but much cheaper expression. Since $K_{xx} = LL^\top$, we

have $K_{xx}^{-1} = L^{-\top} L^{-1}$, so

$$K_{xs}^{\top} K_{xx}^{-1} K_{xs} = K_{xs}^{\top} L^{-\top} L^{-1} K_{xs} = (L^{-1} K_{xs})^{\top} (L^{-1} K_{xs}) = v^{\top} v,$$

where $v = L^{-1} K_{xs}$ is exactly what `solve_triangular` computes (solving $Lv = K_{xs}$, which is fast because L is triangular). This way we never build K_{xx}^{-1} at all — not for the mean, and not for the covariance — we only ever solve triangular systems against the one factor L we already computed.

- **Why bother, for a 9×9 matrix?** At this size it genuinely doesn't matter for speed or accuracy — `np.linalg.inv` would have given essentially the same numbers. We switched anyway because this is the pattern that actually matters once n (the number of training points) grows: forming K_{xx}^{-1} explicitly is both an $O(n^3)$ operation *and* a numerically riskier one (its condition number is the square of K_{xx} 's condition number), while factorizing once and solving twice is also $O(n^3)$ for the factorization but each subsequent solve is only $O(n^2)$ and much better conditioned. Writing it this way from the start meant we wouldn't have to rewrite this cell later if we ever plugged in a larger training set.

2.3 Why we add jitter again before sampling from the posterior

```
1 posterior_samples = np.random.multivariate_normal(  
2     mu_post,  
3     cov_post + 1e-8*np.eye(len(x)),  
4     size=5  
5 )
```

Same reasoning as Task 1: `cov_post` is a difference of two matrices ($K_{**} - K_{*n} K_{nn}^{-1} K_{n*}$), and subtracting two large, similar matrices from each other is exactly the kind of operation that produces tiny negative eigenvalues from rounding error even when the true result is positive semi-definite. We jitter it right before the one place it's actually used for sampling, rather than modifying `cov_post` itself, so that the printed/plotted standard deviations (`std = np.sqrt(np.diag(cov_post))`) reflect the real posterior variance rather than the artificially inflated jittered version.

3 Task 3 — MCMC Code

3.1 Why the prior check returns early, before touching any matrix

```
1 def log_prior_lengthscale(l):  
2     if l <= 0:  
3         return -np.inf  
4     shape, scale = 2.0, 1.0  
5     return (shape - 1) * np.log(l) - (l / scale)
```

The `if l <= 0` check comes first and returns immediately, before any $\log(l)$ is evaluated. This matters for two reasons: first, $\log(l)$ for $l \leq 0$ is either undefined or complex, so evaluating it would either crash or silently produce `nan`; second, by returning `-np.inf` immediately, we let the calling function (`log_posterior_lengthscale`) short-circuit before ever building the (potentially large) covariance matrix for an invalid ℓ , which is both correct and avoids wasted computation.

3.2 Why we bail out of the posterior before building the kernel, and why we wrap the linear algebra in try/except

```
1 def log_posterior_lengthscale(l, x_train, y_train, sigma=1.0, noise_std=0.2):
2     log_prior = log_prior_lengthscale(l)
3     if np.isinf(log_prior):
4         return -np.inf
5
6     K = rbf_kernel(x_train, x_train, sigma=sigma, length_scale=l) \
7         + (noise_std**2) * np.eye(len(x_train))
8
9     try:
10         sign, logdet = np.linalg.slogdet(K)
11         if sign <= 0:
12             return -np.inf
13         K_inv = np.linalg.inv(K)
14
15         n = len(y_train)
16         log_likelihood = -0.5 * y_train.T @ K_inv @ y_train \
17             - 0.5 * logdet - 0.5 * n * np.log(2 * np.pi)
18
19         return log_likelihood + log_prior
20
21     except np.linalg.LinAlgError:
22         return -np.inf
```

A few deliberate choices here:

- **Checking the prior first, and returning immediately if it's `-inf`:** this mirrors the early-exit logic in `log_prior_lengthscale` itself — there's no point building an $n \times n$ covariance matrix and doing a determinant/inverse for a length-scale that the prior has already ruled out.
- **`np.linalg.slogdet` instead of `np.log(np.linalg.det(K))`:** for larger matrices, `det(K)` can itself underflow to 0 or overflow, and taking `log` of that gives `-inf` or garbage even when the true log-determinant is a perfectly normal number. `slogdet` computes the sign and the log of the absolute value separately and more stably, which is why it returns a `(sign, logdet)` pair instead of one number.
- **Checking `sign <= 0`:** for a valid covariance matrix the determinant must be strictly positive, so `sign` should be `+1`. If it comes back 0 or `-1`, that's a signal the matrix isn't actually positive definite (which can happen for pathological length-scales), so we treat that proposal as having zero posterior probability rather than trusting a nonsensical `logdet` value.
- **The `try/except np.linalg.LinAlgError` around the whole block:** even after the `sign` check, `np.linalg.inv` can still fail outright on a near-singular matrix. Rather than letting the whole MCMC chain crash on one bad proposal (which would happen many, many iterations into a 15,000-step run), we catch that specific exception and treat it the same way as an invalid prior: reject by returning `-np.inf`. We deliberately catch only `LinAlgError` and not a bare `except:`, so that unrelated bugs elsewhere in the function still surface as real errors instead of being silently swallowed.

3.3 Why the acceptance step is written as a log-space comparison

```
1 def metropolis_hastings(n_samples, initial_l, proposal_width=0.15):
2     samples = []
3     current = initial_l
4     current_logp = log_posterior_lengthscale(current, x_train, y_train)
5
6     for i in range(n_samples):
7         proposal = current + proposal_width * np.random.randn()
8         proposal_logp = log_posterior_lengthscale(proposal, x_train, y_train)
9
10        log_alpha = proposal_logp - current_logp
11
12        if np.log(np.random.rand()) < log_alpha:
13            current = proposal
14            current_logp = proposal_logp
15
16        samples.append(current)
17
18    return np.array(samples)
```

A few things worth calling out:

- **current_logp is computed once before the loop, then updated only on acceptance, instead of being recomputed from scratch every iteration.** We already have proposal_logp computed for the acceptance test, so if we accept, we can just reassign `current_logp = proposal_logp` instead of calling `log_posterior_lengthscale(current, ...)` again — that would be a second, redundant, expensive matrix inverse and determinant on every single iteration for no benefit.
- **log_alpha = proposal_logp - current_logp instead of computing a ratio and comparing to a threshold in probability space.** Since $p(\ell)$ values here routinely involve exponentials of moderately large negative numbers, computing the actual ratio $p(\ell')/p(\ell)$ risks dividing very small numbers by very small numbers (or underflowing to exact 0), which would make the acceptance decision unreliable. Working entirely in log-space and comparing $\log(U) < \log \alpha$ (where $U \sim \text{Uniform}(0,1)$) is mathematically identical to comparing $U < \alpha = \min(1, p(\ell')/p(\ell))$, but never actually computes α itself, so it never has an underflow/overflow problem in the first place. (Technically we compare against $\log \alpha$ directly, not $\min(1, \alpha)$ clipped to a probability — but since $\log(U) < 0$ always and any $\log \alpha \geq 0$ trivially satisfies the comparison, this is equivalent to the usual $\min(1, \alpha)$ acceptance rule without needing an explicit min.)
- **No Hastings correction term appears anywhere.** That's because our proposal, $\ell' = \ell + \mathcal{N}(0, \text{width}^2)$, is symmetric: the probability density of proposing ℓ' from ℓ is identical to the density of proposing ℓ from ℓ' . For a symmetric proposal, the general Metropolis-Hastings ratio $\frac{p(\ell')q(\ell|\ell')}{p(\ell)q(\ell'|\ell)}$ reduces to the plain Metropolis ratio $\frac{p(\ell')}{p(\ell)}$ because the q terms cancel — which is exactly the `log_alpha` line above. If we had used an asymmetric proposal (e.g. a log-normal step), we would have needed to add the log of the proposal-density ratio here as well.
- **samples.append(current) happens unconditionally, every iteration,** not only on acceptance. This is intentional: if a proposal is rejected, the chain is supposed to stay at its

current value for that iteration (that’s literally what makes it a valid Markov chain with the right stationary distribution) — so we record `current` regardless of whether it just changed or not, rather than only recording accepted values (which would silently throw away the correct relative frequencies of the chain).

3.4 Why burn-in is a simple array slice

```
1 burn_in = int(0.2 * len(chain))
2 plt.hist(chain[burn_in:], bins=50, density=True, ...)
```

We used a plain slice, `chain[burn_in:]`, rather than any special data structure, because burn-in removal is nothing more than “throw away a prefix of the array” — the chain is stored as a single ordered numpy array from `metropolis_hastings`, so slicing off the first 20% is both the simplest and the fastest way to discard the samples collected before the chain had settled near its stationary distribution. Computing `burn_in` as `int(0.2 * len(chain))` rather than hardcoding a fixed number of iterations (like 3000) also means the burn-in fraction stays correct even if `n_samples` in the call to `metropolis_hastings` is changed later.

4 Task 4 — Langevin Dynamics Code

4.1 Why the score function uses a numerical derivative instead of an analytic one

```
1 def log_pi(x):
2     return np.log(0.5*np.exp(-0.5*(x+2)**2) + 0.5*np.exp(-0.5*(x-2)**2))
3
4 def score(x):
5     eps = 1e-5
6     return (log_pi(x + eps) - log_pi(x - eps)) / (2 * eps)
```

We could have worked out $\nabla \log \pi(x)$ by hand and hardcoded the closed-form derivative. Instead we used a central finite difference, $(\log \pi(x + \varepsilon) - \log \pi(x - \varepsilon)) / (2\varepsilon)$, mainly so that `score(x)` stays correct automatically if `log_pi` is ever changed to a different target distribution — we don’t have to re-derive and re-type a new gradient by hand every time we experiment with a different `log_pi`. We used the *central* difference (evaluating at both $x + \varepsilon$ and $x - \varepsilon$) rather than a one-sided forward difference $(\log \pi(x + \varepsilon) - \log \pi(x))$ because the central version has $O(\varepsilon^2)$ error instead of $O(\varepsilon)$, which matters over 20,000 Langevin steps where small per-step gradient errors could otherwise accumulate. We picked $\varepsilon = 10^{-5}$ as a standard middle-ground value: small enough that the finite-difference approximation is accurate, but not so small that floating-point subtraction of two very close `log_pi` values starts losing precision.

4.2 Why the Euler–Maruyama update is written as two separate lines before the update, not one

```
1 def langevin_dynamics(n_steps=20000, eps=0.02, x0=0.0):
2     x = x0
3     samples = []
4
5     for _ in range(n_steps):
```



```

6     drift = 0.5 * eps * score(x)
7     noise = np.sqrt(eps) * np.random.randn()
8     x = x + drift + noise
9     samples.append(x)
10
11     return np.array(samples)

```

We deliberately computed `drift` and `noise` as two separate named variables before combining them, instead of writing the update as one dense expression like `x = x + 0.5*eps*score(x) + np.sqrt(eps)*np.random.randn()`. This isn't just style: it made it much easier, while debugging, to print or plot `drift` and `noise` separately to check they were each the right order of magnitude (drift should shrink as x moves toward a mode, since `score(x)` shrinks near a peak of $\log \pi$; noise should stay roughly constant in scale throughout). The naming also documents the SDE decomposition directly in the code: anyone reading this function can immediately match `drift` to the $\frac{1}{2}\nabla \log \pi(X_t) dt$ term and `noise` to the dW_t term without needing to mentally parse a one-liner.

One detail that's easy to get wrong here: `noise` scales with `np.sqrt(eps)`, not `eps`. This isn't a stylistic choice, it's required by the math — a Brownian increment over a time step of size `eps` has variance `eps`, so its standard deviation (which is what `np.random.randn()` needs to be scaled by) is `np.sqrt(eps)`. Writing `eps * np.random.randn()` instead would silently make the noise far too small once `eps` is small, and the sampler would basically stop exploring and get stuck near whichever mode it started closest to.

4.3 Why we accumulate into a Python list and convert to an array at the end

```

1 samples = []
2 for _ in range(n_steps):
3     ...
4     samples.append(x)
5 return np.array(samples)

```

For 20,000 steps, pre-allocating a numpy array (`np.empty(n_steps)`) and writing into it by index would be marginally faster than repeated `list.append`, but the difference is invisible at this scale, and appending to a plain Python list keeps the loop body simpler to read and modify while we were still experimenting with the step size and number of steps. Converting to `np.array(samples)` once at the very end, rather than working with a Python list throughout, means every downstream computation (histogram, slicing off burn-in, computing the empirical mean/variance) gets to use fast, vectorized numpy operations instead of list comprehensions.

4.4 Why the true density is explicitly renormalized before plotting

```

1 grid = np.linspace(-6,6,300)
2 true_density = np.exp([log_pi(x) for x in grid])
3 true_density /= np.trapezoid(true_density, grid) # Normalize

```

`log_pi(x)` was defined as an *unnormalized* log-density on purpose — the 0.5 weights on each Gaussian component happen to make it integrate to 1 analytically in this particular case, but we didn't want to rely on that being true by coincidence every time someone tweaks the mixture weights or component variances. So before overlaying `true_density` on the histogram (which `plt.hist(..., density=True)` normalizes to integrate to 1 by construction), we explicitly normalize it ourselves using the trapezoidal rule, `np.trapezoid(true_density, grid)`, over the same

grid it was evaluated on. Without this line, if the density weights were ever changed, the red curve and the blue histogram in the comparison plot would silently stop being on the same scale, and a mismatch there could look like a bug in the sampler when it was really just a missing normalization.

Closing Note

Looking back across all four tasks, a pattern shows up repeatedly in our code: whenever we do a numerically risky operation (matrix inverse, determinant, log, division, square root), we either avoid it outright by solving a linear system instead of inverting (Task 2's move from `np.linalg.inv` to `cho_factor/cho_solve`), guard it with an early-exit check (Task 3's `l <= 0` and `sign <= 0` checks), wrap it in a `try/except` (Task 3's `LinAlgError` handling), or nudge the inputs to keep it well-behaved (the jitter terms in Tasks 1 and 2). None of these are things the underlying math *requires* in exact arithmetic — they exist entirely because floating-point computation isn't exact, and writing the code to anticipate that turned out to matter more for getting a working final result than getting the formulas themselves right. The Task 2 rewrite in particular is a good example of this: the naive and Cholesky-based formulas are mathematically identical, but only one of them stays well-behaved as n grows.